PHASE 1 · AGENT GIT (CLOUD)

Phase 1·Git — Packing engine, 3D load plan, map style

You can show after it: Any trip gets a 3D plan with reasons for anything that didn't fit; “Use this plan” activates it; the map style is ready for Phase 2.

RUN WITH	BRANCH	TIME · MODEL
Agent Git — claude.ai/code, cloud (Account 2)	`git/phase1-load-plan`	≈ 3 h · Opus for the engine, Sonnet for the rest

Starts after	Runs at the same time as	Builds (Architecture section 8)
Phase 0 (tag v0)	1·Code — Trucks, trips, cargo	F10 Packing engine F11 3D load plan F12 Dark map style

1. Your lane and the other agent's lane

	You — 1·Git	Other agent — 1·Code
Builds	F10 Packing engine F11 3D load plan F12 Dark map style	F7 Trucks F8 Trips and stops F9 Cargo list
Writes only in	`src/features/load-planner`, `public/map`, `tools/map-preview.html`.	`src/features/{trucks,trips,cargo}`, `supabase/sql/phase1code_*.sql`.

Never touch: `src/features/{trucks,trips,cargo}` (Agent Code is building them). Read trucks, stops and cargo through your own `load-planner/api.ts` using the committed types. Need something from the other lane? Use the fallback in your build plan or write a CONTRACT CHANGE REQUEST (Engineering Reference 12.4).

2. Already built when you start

- Project, database (+ real POIs), contracts, module + slot system, router.
- Sign-in pages, accounts, demo trucks for every new company.
- Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView.
- Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`.

Read before planning: CLAUDE.md, docs/PROGRESS.md, the Plan PDF (sections 0, 1, 4 and your features in 8), Design System sections 2–6, 7.7, 8, 9, 13, Engineering Reference sections 1, 2, 3, 7, 8, and the code in `src/app`, `src/components/common`, `src/types/domain.ts` and the `index.ts` of features you use.

3. Start the session

1. Open **claude.ai/code** in the browser (Account 2 — keeps its limit separate from Agent Code).

2. Repository `spareway`, branch `main`, environment `spareway-cloud`, **Plan mode**, model as shown, paste the prompt.

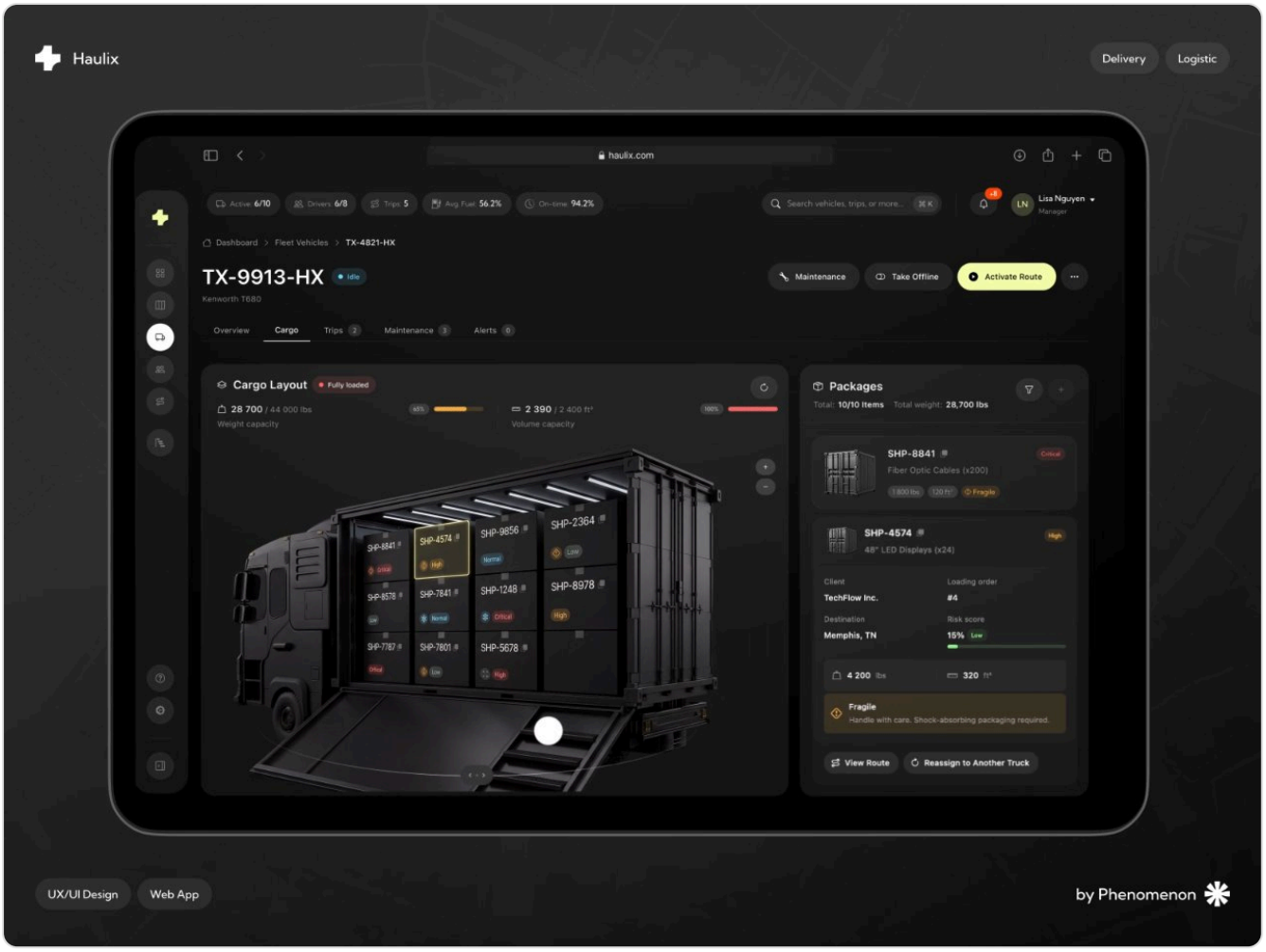
```
You are Agent Git (cloud session) for Spareway. Start Phase 1·Git. Read
docs/Spareway_01_Plan_Architecture_and_Spec.pdf, docs/Spareway_02_Design_System.pdf and
docs/Spareway_03_Engineering_Reference.pdf (all pages, in parts) and
docs/phases/Phase1_Git_Packing_Engine_3D_Plan_and_Map_Style.pdf. Also read CLAUDE.md and
docs/PROGRESS.md. Create branch git/phase1-load-plan from main and push only there. You
have no Supabase, secrets or real phone: never try to connect; use the committed types,
fixtures and mocks. Ask me your questions first, then show your plan. Do not write code
before I approve the plan.
```

4. Questions Claude must ask first

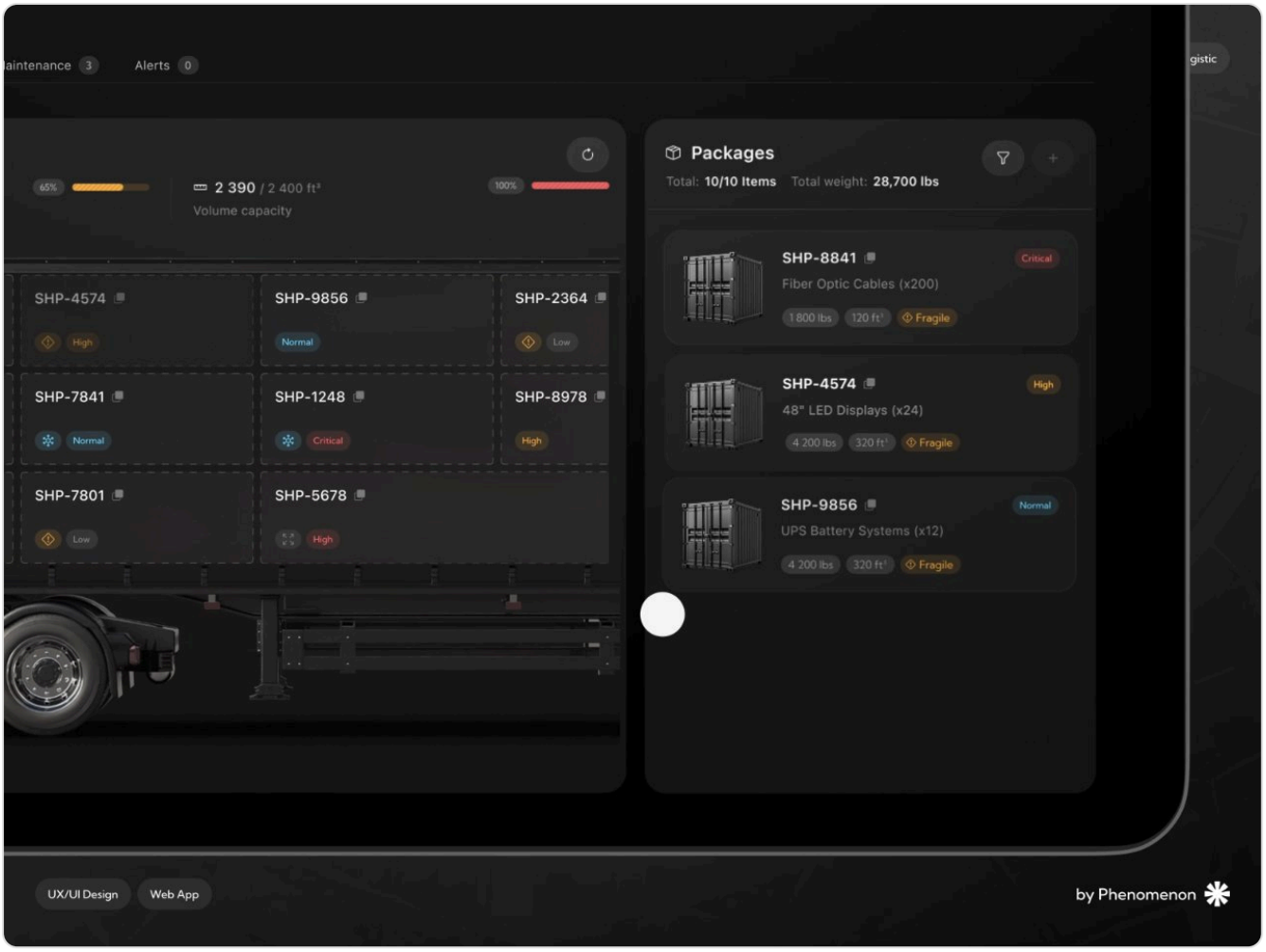
1. Stacking: a box's whole base must rest on boxes below (default) or allow some overhang?
2. Heavier boxes only on boxes at least as heavy (default)?
3. On phone: 3D first with lists below (default) or top view first?

5. Build plan (in this order)

1. **Engine + tests first** (F10) with the demo-trip fixture (Engineering Reference 7) — don't continue until they pass.
2. **Worker + hooks**: usePacker, usePlanForTrip (reuse same hash, save, activate, realtime progress) with mocked api + worker tests.
3. **3D load plan** (F11) as Design System 7.7 with fixture mode; plug into `trip.steps`, `home.actions`, `truck.tabs`.
4. **Map style** (F12) + icons + preview page.
5. Write the “Code must verify” list; Hindi; push.



Haulix reference 4 — Vehicle detail with 3D truck Cargo Layout + Packages detail — the target: 3D truck, Cargo Layout card, Packages panel.



Haulix reference 5 — Cargo grid + Packages list close-up — cards, chips and bars.

6. Check before saying done

- ☐ `npm run check` and `npm run build` pass with zero errors.
- ☐ Engine tests pass (demo trip 104 placed, 0 unplaced, < 1.5 s; over-payload fixture gives the right reason).
- ☐ Hook tests pass; plan renders in fixture mode.
- ☐ Map style JSON loads in a test.
- ☐ Design checklist by code review; “Code must verify” list written.
- ☐ Every new screen tested with the Hindi button (labels + demo data).
- ☐ Design System section 13 check done.

7. Hand-off

1. Update the Phase 1·Git section of `docs/PROGRESS.md` (Engineering Reference 12.2): built, how to show it, open points.
2. Write the **“Code must verify”** list: what you couldn't test without the database, keys or a phone — exact clicks and expected results.
3. Add assumptions to `docs/DECISIONS.md`.
4. Push `git/phase1-load-plan` (do not merge to main).
5. Tell the user “Phase 1·Git ready to merge” + the 3 most important things to check.